



DEPARTAMENTO DE INGENIERÍA ELÉCTRICA
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE CONCEPCIÓN
CONCEPCIÓN, CHILE.



549452-1 - Procesamiento Digital De Imágenes

MiniProyecto 1

Profesor: Vincenzo Caro Fuentes

Integrantes: Benjamín Andrés Ceballo Gil

Christian Daniel Silva Pinilla

6 de octubre de 2026

Concepción, 6 de octubre de 2026

Índice

1. Introducción	1
2. Desarrollo	2
2.1. Item 1.1: Adquisición y construcción del dataset	2
2.2. Item 1.2: Transformaciones e Histogramas	4
2.2.1. Código Transformaciones	4
2.2.2. Resultados e Histogramas	8
2.2.3. Histogramas: Estiramiento y Ecualización	12
2.3. Item 1.3: Segmentación De La Estructura De Interés	16
2.3.1. Código Segmentación	16
2.3.2. Resultados Segmentación RGB	20
2.3.3. Resultados Segmentación Escala De Grises	21
2.3.4. Resultados Segmentación HSV	22
2.3.5. Resultados Segmentación CIELAB	23
2.3.6. Segmentación Por IA: Código	24
2.3.7. Segmentación Por IA: Resultados	25
2.3.8. Segmentación Por IA: Comparación IA y Manual	27
3. Resumen Procedimiento General	32
3.1. Diagrama de Bloques Item 1.2	32
3.2. Diagrama de Bloques Item 1.3	32
3.3. Tabla Resumen	32
4. Conclusión	33

1. Introducción

El presente informe detalla el análisis y segmentación de huellas dactilares bajo tres condiciones de adquisición distintas: normal (seco), con agua (mojado) y con crema (untado). Para llevar a cabo este estudio, se desarrolló un entorno de procesamiento en Python utilizando las librerías OpenCV y Matplotlib. Inicialmente, se evalúa el comportamiento visual y estadístico de las capturas en múltiples espacios de color, específicamente RGB, Escala de Grises, HSV y CIELAB. Sobre estos espacios, se aplican transformaciones de intensidad, tales como el Estiramiento y la Ecualización, con el objetivo de maximizar el contraste y destacar las características morfológicas propias de cada medio.

Posteriormente, se procede a aislar la estructura de interés mediante técnicas de umbralización global (Binaria y Otsu) complementadas con operaciones morfológicas. Finalmente, la robustez de este procedimiento clásico se contrasta contra un modelo de Inteligencia Artificial basado en agrupamiento no supervisado (K-Means). El desempeño comparativo de ambas metodologías se evalúa de manera cuantitativa utilizando métricas de solapamiento espacial (IoU, Coeficiente de Dice) y métricas de error global (MAE, PSNR), permitiendo identificar las vulnerabilidades de los algoritmos frente a gradientes de iluminación severos.

2. Desarrollo

2.1. Item 1.1: Adquisición y construcción del dataset

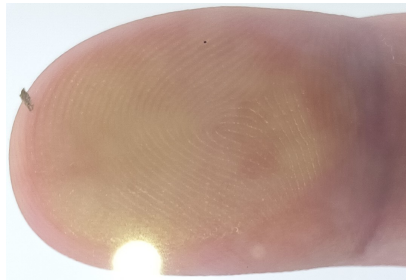


Para nuestro proyecto, nos asignaron imágenes de la yema del dedo índice, con el objetivo de obtener la huella dactilar, en diferentes medios:

1. Normal (Seco)
2. Agua (Mojado)
3. Crema (Untado)

Y en diferentes ángulos (Normal y Lado), para dos dedos índices, del compañero 1 y del compañero 2. Las fotos que capturamos son las siguientes:

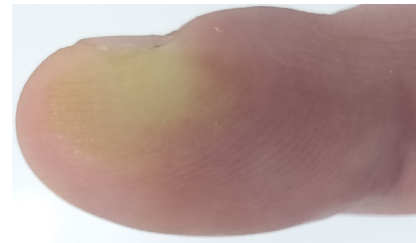
Las imágenes anteriores fueron recortadas para centrar la huella dactilar (para evitar el uso del comando *roi* en el código).



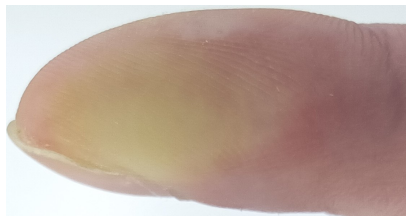
(a) Dedo 1 (Normal, Seco)



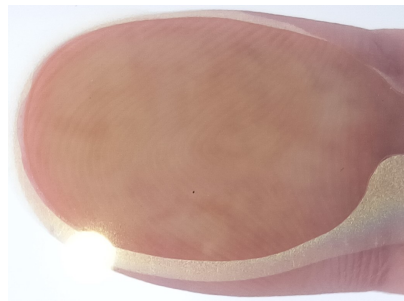
(b) Dedo 2 (Normal, Seco)



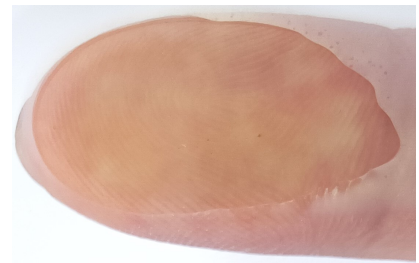
(c) Dedo 1 (Lado, Seco)



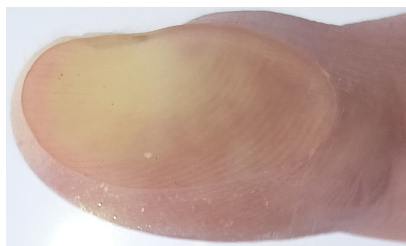
(d) Dedo 2 (Lado, Seco)



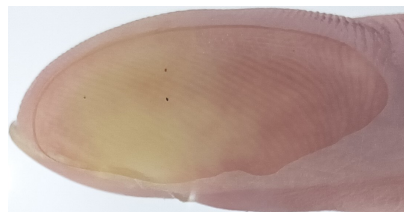
(e) Dedo 1 (Normal, Agua)



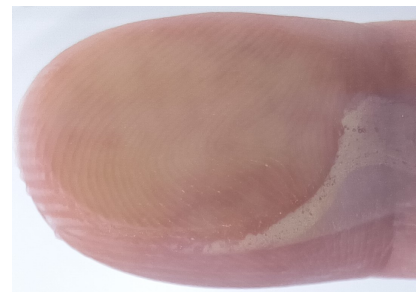
(f) Dedo 2 (Normal, Agua)



(g) Dedo 1 (Lado, Agua)



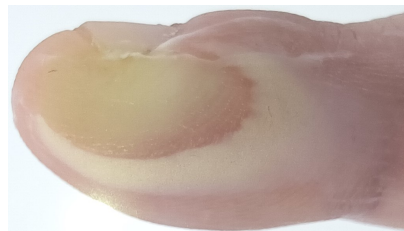
(h) Dedo 2 (Lado, Agua)



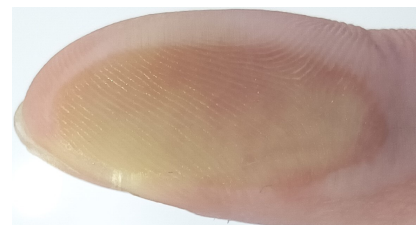
(i) Dedo 1 (Normal, Crema)



(j) Dedo 2 (Normal, Crema)



(k) Dedo 1 (Lado, Crema)



(l) Dedo 2 (Lado, Crema)

Fig. 1: Fotos del Dataset

2.2. Item 1.2: Transformaciones e Histogramas

2.2.1. Código Transformaciones

Nuestro grupo creó un código en Python llamado **visualizador.py** para analizar el espacio de colores que usa las librerías **OpenCV** (para el procesamiento de imágenes) y **Matplotlib** (para generar plots y mostrar las imágenes originales y procesadas al usuario).

El código es el siguiente:

```
import cv2
import matplotlib.pyplot as plt

def visualizar_grupo(rutas_imagenes, grupo_num, modo):
    # Se usan 3 columnas para Escala de Grises y 5 para los espacios multicanal (
    # Original + 3 Canales + Histograma)
    ncols = 3 if modo == 'G' else 5
    # Se aumenta ligeramente el ancho (20, 10) para dar espacio a la quinta columna
    fig, axes = plt.subplots(nrows=3, ncols=ncols, figsize=(20, 10))
    fig.suptitle(f'An lisis Grupo {grupo_num} - Espacio: {modo}', fontsize=16)

    for i, ruta in enumerate(rutas_imagenes):
        # 1. Cargar imagen y convertir a RGB base
        img_bgr = cv2.imread(ruta)
        if img_bgr is None:
            print(f"Advertencia: No se pudo cargar '{ruta}'.")
            continue

        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        # Columna 0: Imagen Original (com n para todos los modos)
        axes[i, 0].imshow(img_rgb)
        axes[i, 0].set_title(f'Img {i + 1 + (int(grupo_num)-1)*3}: Original')
        axes[i, 0].axis('off')

        # 2. L gica de visualizaci n seg n el modo
        if modo == 'G':
            img_gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)

            axes[i, 1].imshow(img_gray, cmap='gray')
            axes[i, 1].set_title('Escala de Grises')
            axes[i, 1].axis('off')

            hist = cv2.calcHist([img_gray], [0], None, [256], [0, 256])
            axes[i, 2].plot(hist, color='black')
            axes[i, 2].set_xlim([0, 256])
            axes[i, 2].set_title('Histograma')
            axes[i, 2].grid(alpha=0.3)

        elif modo == 'RGB':
            r, g, b = cv2.split(img_rgb)

            axes[i, 1].imshow(r, cmap='Reds')
            axes[i, 1].set_title('Canal R')
            axes[i, 1].axis('off')

            axes[i, 2].imshow(g, cmap='Greens')
            axes[i, 2].set_title('Canal G')
            axes[i, 2].axis('off')
```

```

axes[i, 3].imshow(b, cmap='Blues')
axes[i, 3].set_title('Canal B')
axes[i, 3].axis('off')

# Columna 4: Histograma superpuesto de los 3 canales
for canal, color, label in zip([r, g, b], ['red', 'green', 'blue'], ['R',
    'G', 'B']):
    hist = cv2.calcHist([canal], [0], None, [256], [0, 256])
    axes[i, 4].plot(hist, color=color, label=f'Canal {label}')
axes[i, 4].set_title('Histogramas RGB')
axes[i, 4].set_xlim([0, 256])
axes[i, 4].legend(fontsize=8)
axes[i, 4].grid(alpha=0.3)

elif modo == 'HSV':
    img_hsv = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2HSV)
    h, s, v = cv2.split(img_hsv)

    axes[i, 1].imshow(h, cmap='hsv')
    axes[i, 1].set_title('Canal H (Matiz)')
    axes[i, 1].axis('off')

    axes[i, 2].imshow(s, cmap='gray')
    axes[i, 2].set_title('Canal S (Saturación)')
    axes[i, 2].axis('off')

    axes[i, 3].imshow(v, cmap='gray')
    axes[i, 3].set_title('Canal V (Brillo)')
    axes[i, 3].axis('off')

    # Columna 4: Histograma superpuesto de los 3 canales
    for canal, color, label in zip([h, s, v], ['purple', 'cyan', 'black'], ['H',
        'S', 'V']):
        hist = cv2.calcHist([canal], [0], None, [256], [0, 256])
        axes[i, 4].plot(hist, color=color, label=f'Canal {label}')
    axes[i, 4].set_title('Histogramas HSV')
    axes[i, 4].set_xlim([0, 256])
    axes[i, 4].legend(fontsize=8)
    axes[i, 4].grid(alpha=0.3)

elif modo == 'CIELAB':
    # Conversión a CIELAB
    img_lab = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2LAB)
    l_chan, a_chan, b_chan = cv2.split(img_lab)

    axes[i, 1].imshow(l_chan, cmap='gray')
    axes[i, 1].set_title('Canal L (Luminancia)')
    axes[i, 1].axis('off')

    axes[i, 2].imshow(a_chan, cmap='gray')
    axes[i, 2].set_title('Canal A (Verde-Rojo)')
    axes[i, 2].axis('off')

    axes[i, 3].imshow(b_chan, cmap='gray')
    axes[i, 3].set_title('Canal B (Azul-Amarillo)')
    axes[i, 3].axis('off')

    # Columna 4: Histograma superpuesto de los 3 canales
    for canal, color, label in zip([l_chan, a_chan, b_chan], ['black', 'green',
        'blue'], ['L', 'A', 'B']):

```

```

        hist = cv2.calcHist([canal], [0], None, [256], [0, 256])
        axes[i, 4].plot(hist, color=color, label=f'Canal {label}')
        axes[i, 4].set_title('Histogramas CIELAB')
        axes[i, 4].set_xlim([0, 256])
        axes[i, 4].legend(fontsize=8)
        axes[i, 4].grid(alpha=0.3)

plt.tight_layout()
plt.show()

# --- Ejecuci n Principal e Interactiva ---

lista_imagenes = [
    "Fotos_PDI_2026_Grupo1/d1_seco.jpg", "Fotos_PDI_2026_Grupo1/d1_agua.jpg", "
    Fotos_PDI_2026_Grupo1/d1_crema.jpg",
    "Fotos_PDI_2026_Grupo1/d1_lat_seco.jpg", "Fotos_PDI_2026_Grupo1/d1_lat_agua.jpg",
    "Fotos_PDI_2026_Grupo1/d1_lat_crema.jpg",
    "Fotos_PDI_2026_Grupo1/d2_seco.jpg", "Fotos_PDI_2026_Grupo1/d2_agua.jpg", "
    Fotos_PDI_2026_Grupo1/d2_crema.jpg",
    "Fotos_PDI_2026_Grupo1/d2_lat_seco.jpg", "Fotos_PDI_2026_Grupo1/d2_lat_agua.jpg",
    "Fotos_PDI_2026_Grupo1/d2_lat_crema.jpg"
]

modos_validos = ['G', 'RGB', 'HSV', 'CIELAB']
grupos_validos = ['1', '2', '3', '4']

while True:
    print("\nOpciones de comando: [Grupo] [Modo]")
    print("Ejemplos: '1 CIELAB', '3 HSV', '2 G', '4 RGB'")
    entrada = input("Ingresa tu comando (o 'q' para salir): ").strip().upper()

    if entrada in ['Q', 'SALIR', 'EXIT']:
        print("Saliendo del visualizador.")
        break

    partes = entrada.split()

    # Validar que se ingresaron exactamente dos par metros
    if len(partes) != 2:
        print("Error: Debes ingresar el grupo y el canal separados por un espacio.")
        continue

    grupo, modo = partes[0], partes[1]

    if grupo not in grupos_validos:
        print(f"Error: El grupo '{grupo}' no es v lido. Usa 1, 2, 3 o 4.")
        continue

    if modo not in modos_validos:
        print(f"Error: El canal '{modo}' no es v lido. Usa G, RGB, HSV o CIELAB.")
        continue

    # Calcular ndices
    indice = int(grupo) - 1
    inicio = indice * 3
    fin = inicio + 3
    grupo_actual = lista_imagenes[inicio:fin]

    print(f"Generando visualizaci n {modo} para el Grupo {grupo}...")
    visualizar_grupo(grupo_actual, grupo, modo)

```

El código, de forma resumida, permite al usuario ingresar un número del 1 al 4 seguido de **RGB**, **G**, **HSV** o **CIELAB** (por ejemplo: "2 RGB"). El número introducido indica el grupo de imágenes para el procesamiento:

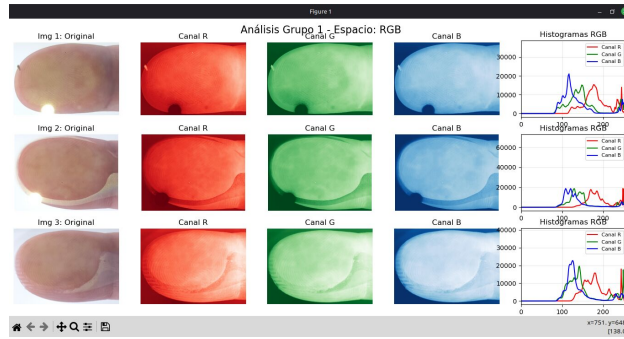
- **1:** Se muestran las imágenes del Grupo 1 (*Dedo 1 Normal Seco*, *Dedo 1 Normal Agua* y *Dedo 1 Normal Crema*).
- **2:** Se muestran las imágenes del Grupo 2 (*Dedo 1 Lado Seco*, *Dedo 1 Lado Agua* y *Dedo 1 Lado Crema*).
- **3:** Se muestran las imágenes del Grupo 3 (*Dedo 2 Normal Seco*, *Dedo 2 Normal Agua* y *Dedo 2 Normal Crema*).
- **4:** Se muestran las imágenes del Grupo 4 (*Dedo 2 Lado Seco*, *Dedo 2 Lado Agua* y *Dedo 2 Lado Crema*).

Mientras que la palabra siguiente permite visualizar las fotografías en diferentes espacios de color y con histogramas incluidos:

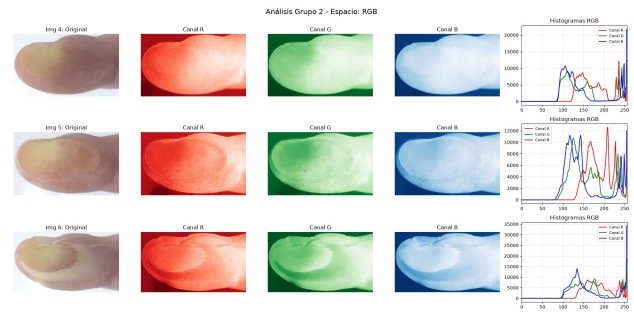
- **RGB:** Las imágenes se separan en los canales *Rojo*, *Verde* y *Azul*.
- **G:** Las imágenes se muestran en *Escala de Grises*.
- **HSV:** Las imágenes se separan en los canales *Matiz*, *Saturación* y *Brillo*.
- **CIELAB:** Las imágenes se separan en los canales *Luminancia*, *Verde-Rojo* y *Azul-Amarillo*.

2.2.2. Resultados e Histogramas

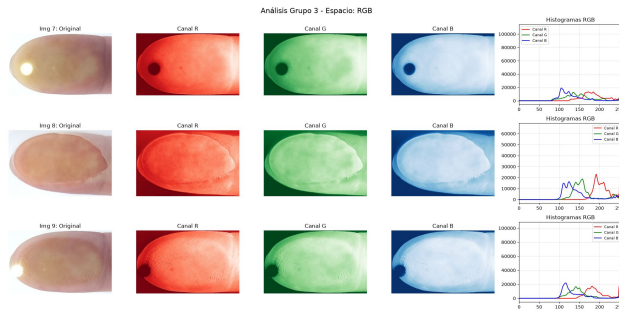
Los resultados de cada espacio de color (del grupo 1 de imágenes, para resumir) son:



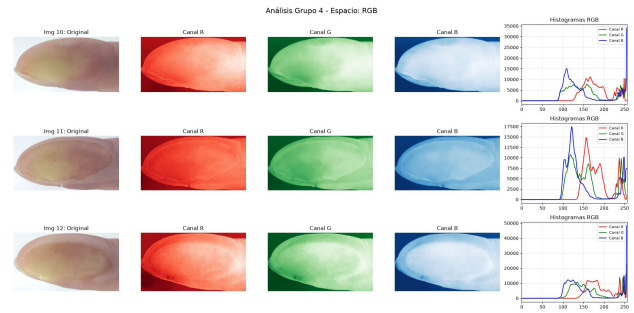
(a) Resultados Grupo 1 RGB



(b) Resultados Grupo 2 RGB



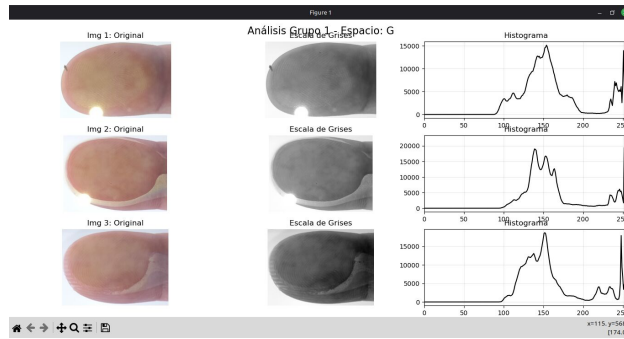
(c) Resultados Grupo 3 RGB



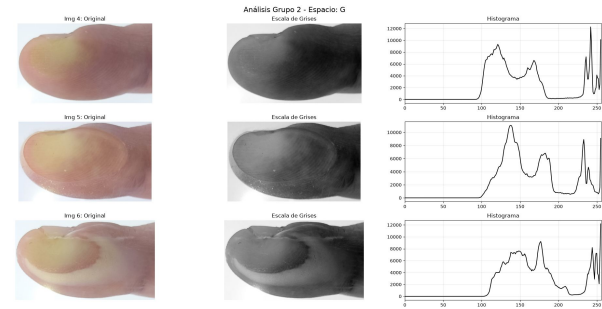
(d) Resultados Grupo 4 RGB

Fig. 2: Resultados Espacio de Color RGB

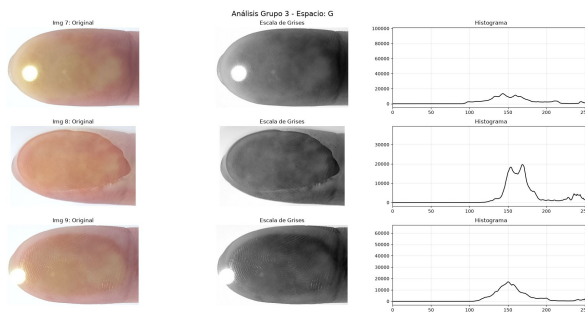
Los resultados del espacio de color RGB son nuestro punto de partida. Los canales Verde y Azul no muestran distinciones entre las condiciones, pero el canal Rojo permite notar diferencias entre condiciones. Hay contraste regular.^{en} los histogramas considerando que los pixeles se concentran a la derecha del histograma (se nota un peak al extremo derecho de los histogramas, se deben a capturar una fuente de luz directamente con la cámara). A pesar de ser condiciones distintas (Seco, Mojado, Untado con Crema), los diferentes histogramas no aportan mucha información visual que permita identificar la condición solo viendo dichos histogramas.



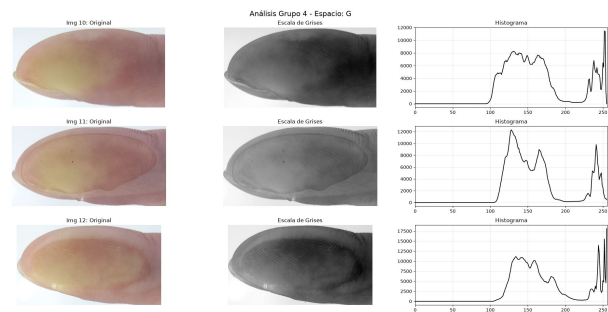
(a) Resultados Grupo 1 Escala De Grises



(b) Resultados Grupo 2 Escala De Grises



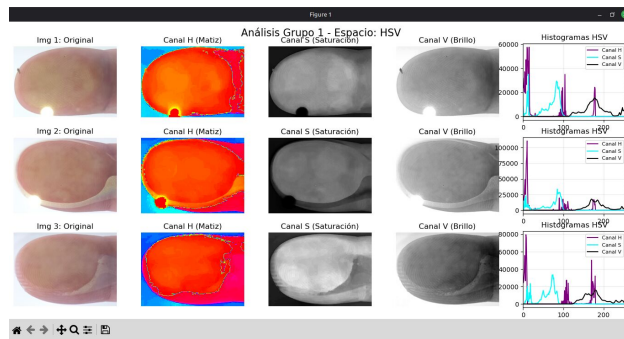
(c) Resultados Grupo 3 Escala De Grises



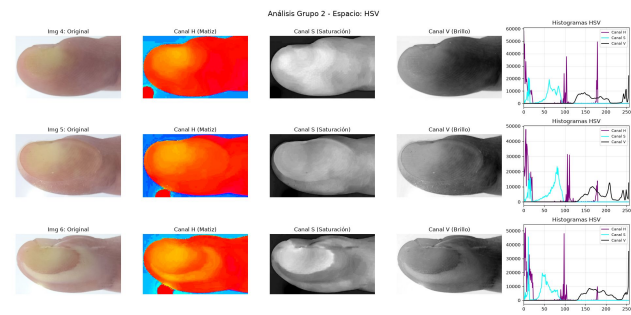
(d) Resultados Grupo 4 Escala De Grises

Fig. 3: Resultados Espacio de Color Escala de Grises

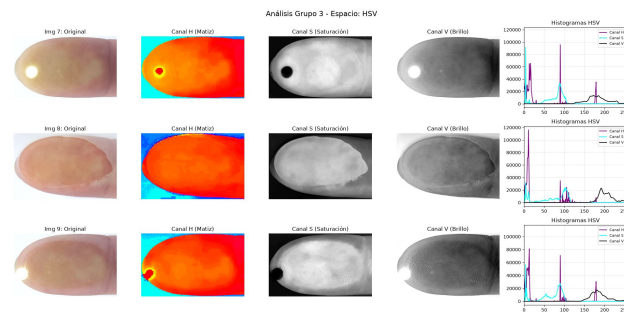
Los resultados del espacio de color Escala de Grises son los más básicos. Tampoco hay demasiado contraste en los histogramas, pues los pixeles se concentran esta vez al centro del histograma. En escala de grises tanto las imágenes como los diferentes histogramas muestran diferencias más notables que el caso de RGB, pero dichas diferencias son insuficientes para identificar la condición solo con dichas imágenes/histogramas.



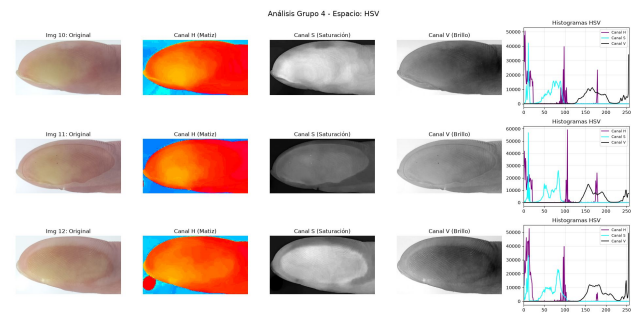
(a) Resultados Grupo 1 HSV



(b) Resultados Grupo 2 HSV



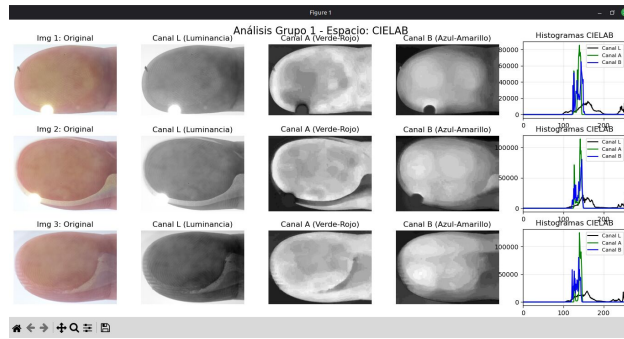
(c) Resultados Grupo 3 HSV



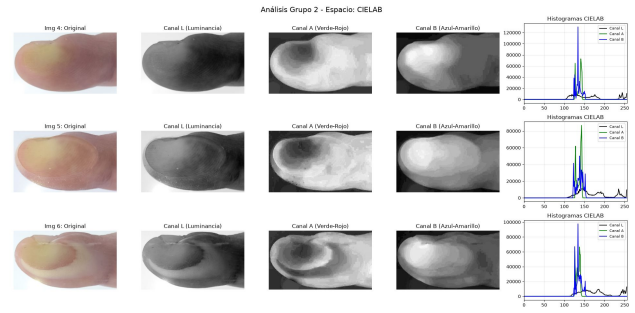
(d) Resultados Grupo 4 HSV

Fig. 4: Resultados Espacio de Color HSV

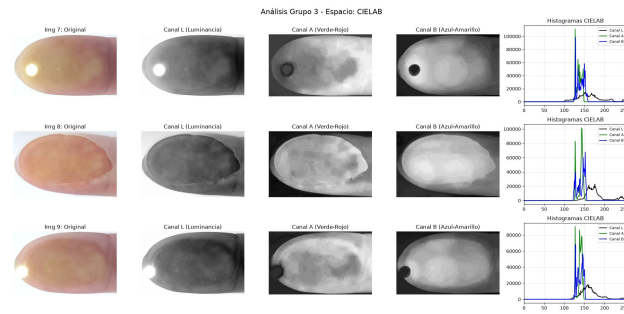
El espacio de color HSV nos entregan histogramas con más información que los casos anteriores. Hay bastante contraste dado que las curvas están distribuidas a lo largo del histograma. Los diferentes histogramas muestran diferencias más notables que el caso de RGB, pero dichas diferencias son insuficientes para identificar la condición solo con dichos histogramas.



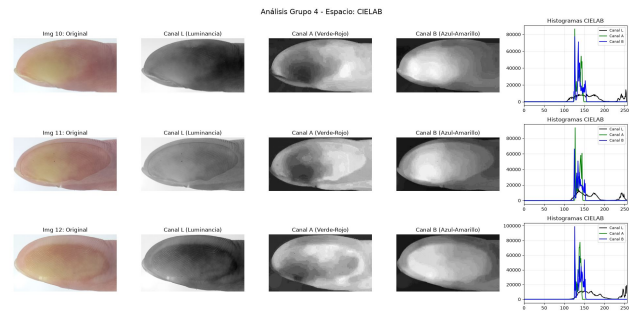
(a) Resultados Grupo 1 CIELAB



(b) Resultados Grupo 2 CIELAB



(c) Resultados Grupo 3 CIELAB



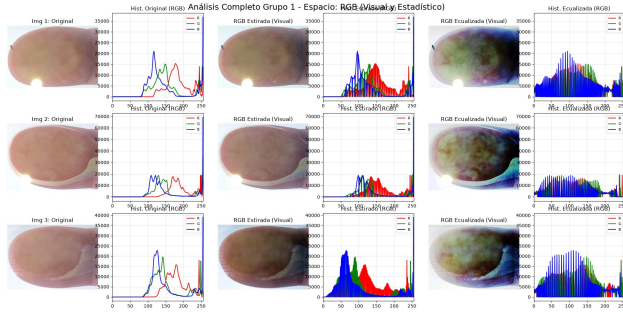
(d) Resultados Grupo 4 CIELAB

Fig. 5: Resultados Espacio de Color CIELAB

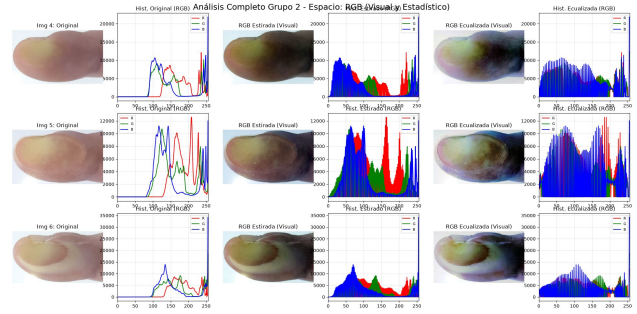
El espacio de color CIELAB muestra diferencias entre las fotos y entre sus histogramas. Hay contraste regular.³¹ Igual que en RGB (los peaks se concentran a la derecha). Notamos que las amplitudes de ciertos canales difieren de acuerdo a las diferentes condiciones.

2.2.3. Histogramas: Estiramiento y Ecualización

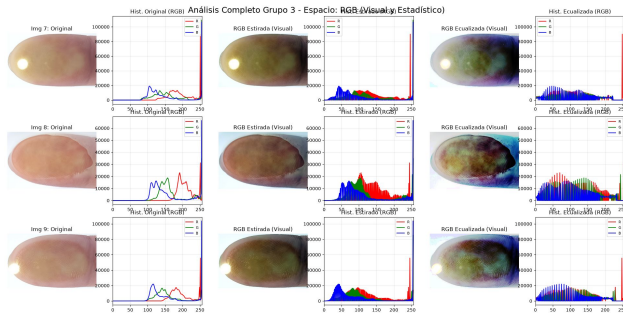
Dados los histogramas anteriores, decidimos realizar las transformaciones **Estiramiento** y **Ecualización**:



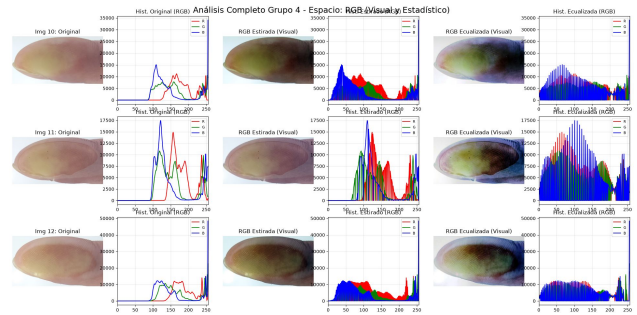
(a) Resultados Grupo 1 RGB



(b) Resultados Grupo 2 RGB



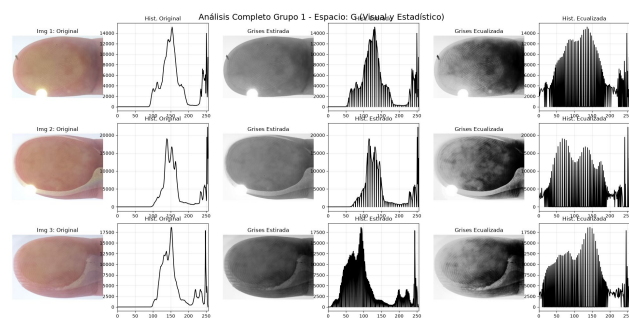
(c) Resultados Grupo 3 RGB



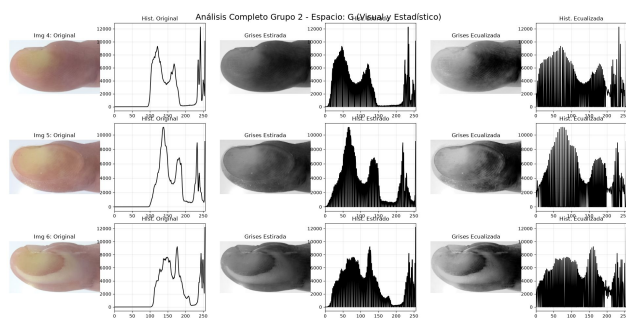
(d) Resultados Grupo 4 RGB

Fig. 6: Resultados Transformaciones Estiramiento y Ecualización (RGB)

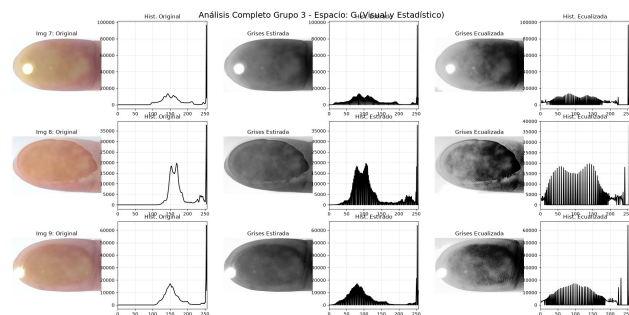
Para RGB, las transformadas no llaman particularmente la atención como para detectar las condiciones.



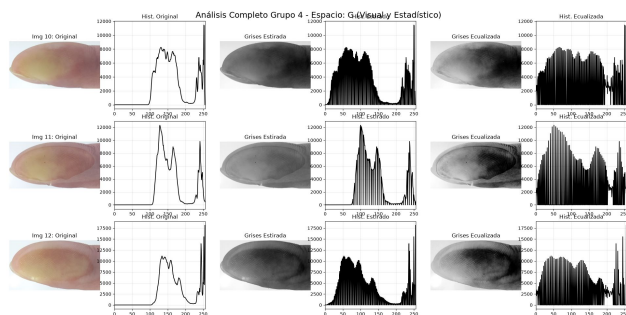
(a) Resultados Grupo 1 RGB



(b) Resultados Grupo 2 Escala De Grises



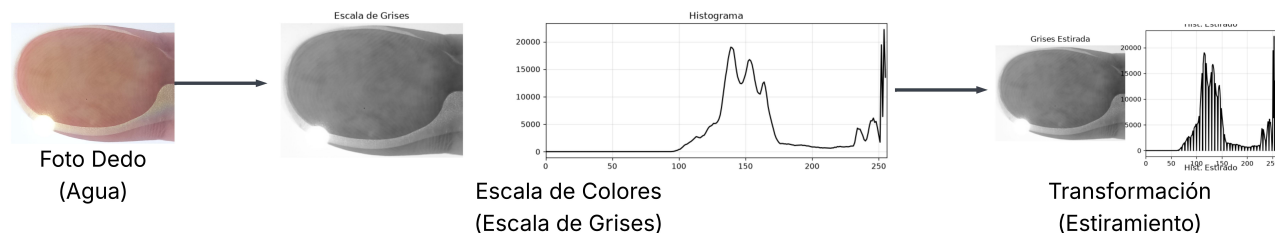
(c) Resultados Grupo 3 Escala De Grises

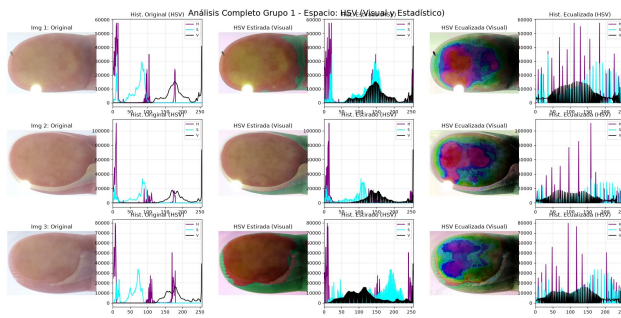


(d) Resultados Grupo 4 Escala De Grises

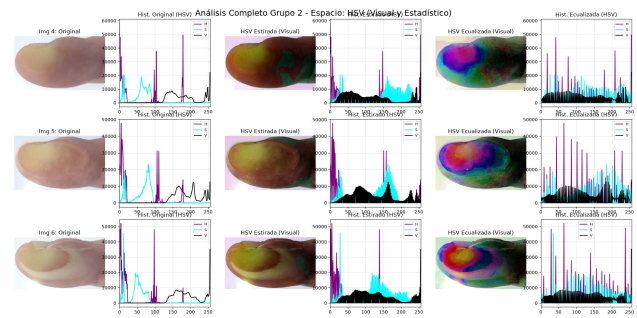
Fig. 7: Resultados Transformaciones Estiramiento y Ecuilización (Escala De Grises)

En el caso de la Escala de Grises, la condición **Dedo Mojado** es la que más destaca en este espacio de color al realizar la transformada de **Estiramiento**. Lo que se vé para la condición de agua, específicamente para el contacto entre el dedo y el vidrio de apoyo, es que existe algo parecido a una elipse la cual corresponde al contorno de agua. Si bien en los otros espacios de colores también destaca la condición mojada con el estiramiento, decidí usar la escala de grises.

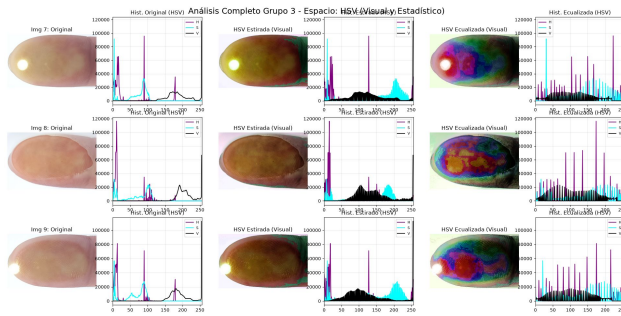




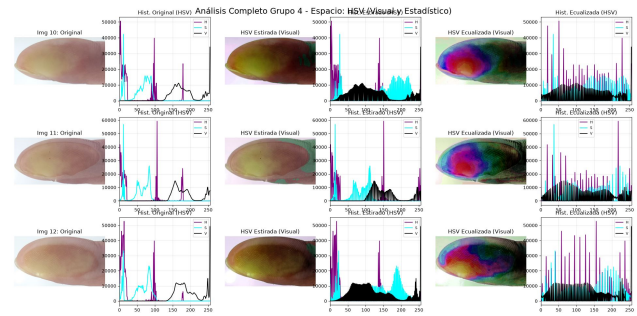
(a) Resultados Grupo 1 HSV



(b) Resultados Grupo 2 HSV



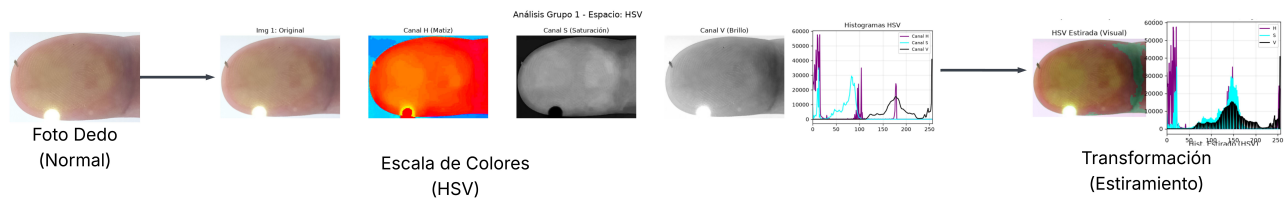
(c) Resultados Grupo 3 HSV

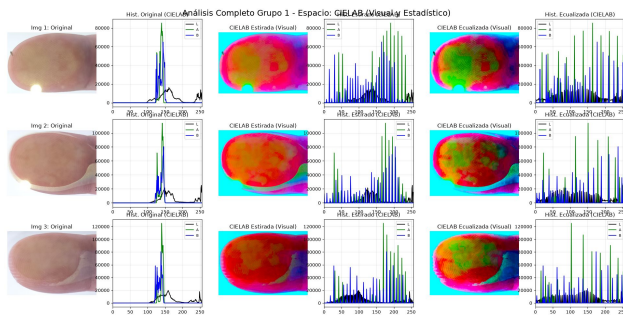


(d) Resultados Grupo 4 HSV

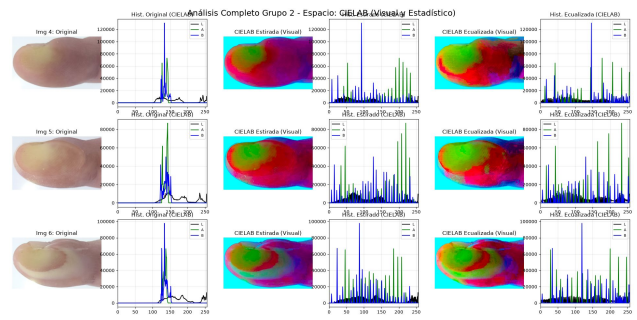
Fig. 8: Resultados Transformaciones Estiramiento y Ecualización (HSV)

Para HSV, decidimos que la transformada de **Estiramiento** es la que más hace destacar la condición **Dedo Normal**, ya que es más cromáticamente uniforme y sin contornos de agua o de crema

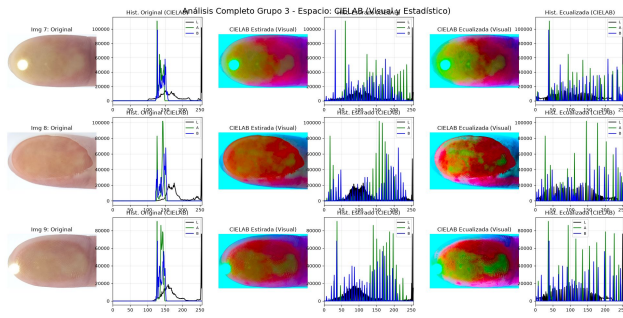




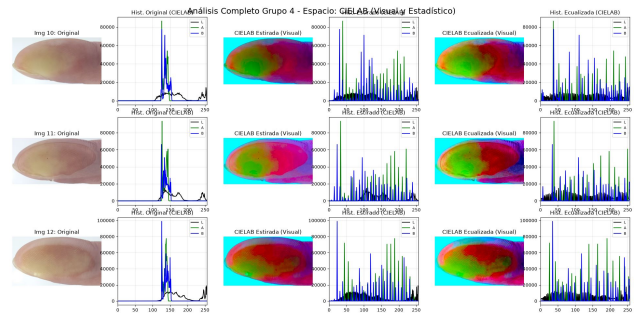
(a) Resultados Grupo 1 CIELAB



(b) Resultados Grupo 2 CIELAB



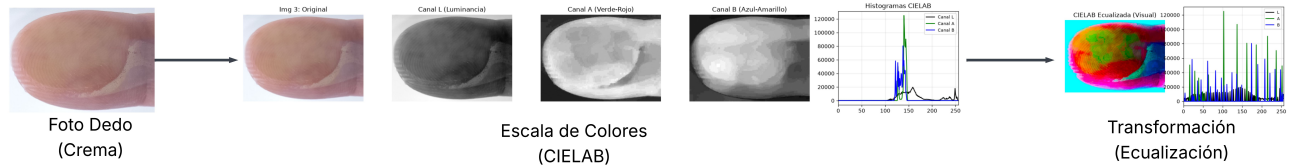
(c) Resultados Grupo 3 CIELAB



(d) Resultados Grupo 4 CIELAB

Fig. 9: Resultados Transformaciones Estiramiento y Ecuación (CIELAB)

Para CIELAB, decidimos que la transformada de **Ecuación** es la que más hace destacar la condición **Dedo Untado con Crema**, ya que se visualizan mejor los contornos donde se acumula la crema.



2.3. Item 1.3: Segmentación De La Estructura De Interés

2.3.1. Código Segmentación

Para el ítem de Segmentación, utilizamos el código:

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# --- Funciones de Transformación ---

def aplicar_estiramiento(canal):
    """Aplica la transformación de estiramiento de contraste."""
    r_a = canal.min()
    r_b = canal.max()
    if r_a == r_b:
        return canal
    estirada = (canal.astype(np.float32) - r_a) * 255 / (r_b - r_a)
    return np.clip(estirada, 0, 255).astype(np.uint8)

def aplicar_ecualizacion(canal):
    """Aplica la ecualización de histograma global."""
    return cv2.equalizeHist(canal)

# --- Funciones de Graficado ---

def graficar_histograma(ax, canales, colores, labels, titulo):
    """Función para graficar histogramas con Matplotlib."""
    for canal, color, label in zip(canales, colores, labels):
        hist = cv2.calcHist([canal], [0], None, [256], [0, 256])
        ax.plot(hist, color=color, label=label)
    ax.set_title(titulo, fontsize=10)
    ax.set_xlim([0, 256])
    if len(labels) > 1:
        ax.legend(fontsize=8)
    ax.grid(alpha=0.3)

def procesar_y_graficar_canal(axes_row, canal, cmap_orig, nombre_canal,
                             transformacion_func, nombre_trans, color_hist):
    """
    Toma un solo canal, le aplica la transformación asignada y luego genera
    las dos estrategias de segmentación (Binaria 127 y Otsu), mostrando los
    resultados en 7 columnas.
    """
    # 1. Aplicar Transformación (Estiramiento o Ecualización)
    canal_trans = transformacion_func(canal)

    # 2. Aplicar Segmentaciones sobre el canal transformado
    _, binaria = cv2.threshold(canal_trans, 127, 255, cv2.THRESH_BINARY)
    _, otsu = cv2.threshold(canal_trans, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    # Col 0: Canal Original
    axes_row[0].imshow(canal, cmap=cmap_orig)
    axes_row[0].set_title(f"Orig. {nombre_canal}", fontsize=11)
    axes_row[0].axis('off')

    # Col 1: Transformación (visualizado en gris para análisis de intensidad)
    axes_row[1].imshow(canal_trans, cmap='gray')
    axes_row[1].set_title(f"{nombre_trans}", fontsize=11)
    axes_row[1].axis('off')
```

```

# Col 2: Histograma Transformado
graficar_histograma(axes_row[2], [canal_trans], [color_hist], [nombre_canal], f"
    Hist. {nombre_trans}")

# Col 3: Segmentación Binaria
axes_row[3].imshow(binaria, cmap='gray')
axes_row[3].set_title("Seg. Binaria (127)", fontsize=11)
axes_row[3].axis('off')

# Col 4: Histograma Binaria (Típicamente dos líneas verticales en 0 y 255)
graficar_histograma(axes_row[4], [binaria], ['black'], ['Bin'], "Hist. Binaria")

# Col 5: Segmentación Otsu
axes_row[5].imshow(otsu, cmap='gray')
axes_row[5].set_title("Seg. Otsu", fontsize=11)
axes_row[5].axis('off')

# Col 6: Histograma Otsu
graficar_histograma(axes_row[6], [otsu], ['black'], ['Otsu'], "Hist. Otsu")

# --- Lógica Principal de Visualización ---

def visualizar_analisis(grupo, modo):
    # Diccionario para mapear el número de grupo al prefijo base del archivo
    bases = {'1': 'd1', '2': 'd1_lat', '3': 'd2', '4': 'd2_lat'}
    base = bases[grupo]
    carpeta = "Fotos_PDI_2026_Grupo1"

    # =====
    # CASO EXCEPCIÓN: RGB (Sin segmentación)
    # =====
    if modo == 'RGB':
        # Se muestran las 3 condiciones originales para el grupo seleccionado
        rutas = [
            f"{carpeta}/{base}_seco.jpg",
            f"{carpeta}/{base}_agua.jpg",
            f"{carpeta}/{base}_crema.jpg"
        ]
        condiciones = ['Seco', 'Agua (Mojado)', 'Crema']

        fig, axes = plt.subplots(nrows=3, ncols=5, figsize=(20, 10))
        fig.suptitle(f'Análisis Grupo {grupo} - Espacio: RGB (Fotos Originales)',
            fontsize=16)

        for i, (ruta, condicion) in enumerate(zip(rutas, condiciones)):
            img_bgr = cv2.imread(ruta)
            if img_bgr is None:
                print(f"Advertencia: No se pudo cargar '{ruta}'.")
                continue

            img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
            r, g, b = cv2.split(img_rgb)

            axes[i, 0].imshow(img_rgb)
            axes[i, 0].set_title(f'Original: {condicion}')
            axes[i, 0].axis('off')

            axes[i, 1].imshow(r, cmap='Reds')
            axes[i, 1].set_title('Canal R')

```

```

        axes[i, 1].axis('off')

        axes[i, 2].imshow(g, cmap='Greens')
        axes[i, 2].set_title('Canal G')
        axes[i, 2].axis('off')

        axes[i, 3].imshow(b, cmap='Blues')
        axes[i, 3].set_title('Canal B')
        axes[i, 3].axis('off')

        graficar_histograma(axes[i, 4], [r, g, b], ['red', 'green', 'blue'], ['R',
            'G', 'B'], f'Hist. RGB ({condicion})')

plt.tight_layout()
plt.show()

# =====
# CASOS CON TRANSFORMACION Y SEGMENTACION
# =====
else:
    # Configurar la ruta y la transformacion segun la palabra clave ingresada
    if modo == 'HSV':
        condicion = 'seco'
        transformacion_func = aplicar_estiramiento
        nombre_trans = "Estirado"

    elif modo == 'CIELAB':
        condicion = 'crema'
        transformacion_func = aplicar_ecualizacion
        nombre_trans = "Ecualizado"

    elif modo == 'G':
        condicion = 'agua'
        transformacion_func = aplicar_estiramiento
        nombre_trans = "Estirado"

    ruta = f"{carpeta}/{base}_{condicion}.jpg"
    img_bgr = cv2.imread(ruta)

    if img_bgr is None:
        print(f"Error: No se pudo cargar la imagen '{ruta}'. Verifica que exista.")
        return

    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    # Procesamiento para Escala de Grises (1 sola fila)
    if modo == 'G':
        fig, axes = plt.subplots(nrows=1, ncols=7, figsize=(24, 4))
        fig.suptitle(f'Grupo {grupo} | Espacio: {modo} (Dedo {condicion}) |
            Transformacion: {nombre_trans}', fontsize=16)

        img_gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
        procesar_y_graficar_canal(axes, img_gray, 'gray', 'Gris',
            transformacion_func, nombre_trans, 'black')

    # Procesamiento para HSV y CIELAB (3 filas, una por canal)
    else:
        fig, axes = plt.subplots(nrows=3, ncols=7, figsize=(26, 12))
        fig.suptitle(f'Grupo {grupo} | Espacio: {modo} (Dedo {condicion}) |

```

```

        Transformaci n: {nombre_trans}', fontsize=16)

    if modo == 'HSV':
        img_espacio = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2HSV)
        cmaps = ['hsv', 'gray', 'gray']
        nombres = ['H', 'S', 'V']
        colores_hist = ['purple', 'cyan', 'black']
    else: # CIELAB
        img_espacio = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2LAB)
        cmaps = ['gray', 'gray', 'gray']
        nombres = ['L', 'A', 'B']
        colores_hist = ['black', 'green', 'blue']

    canales = cv2.split(img_espacio)

    for i, canal in enumerate(canales):
        procesar_y_graficar_canal(axes[i], canal, cmaps[i], nombres[i],
                                transformacion_func, nombre_trans, colores_hist[i])

    plt.tight_layout()
    plt.show()

# --- Interfaz de Usuario ---

grupos_validos = ['1', '2', '3', '4']
modos_validos = ['G', 'RGB', 'HSV', 'CIELAB']

while True:
    print("\n--- Visualizador Avanzado de Segmentaci n ---")
    print("Opciones de comando: [Grupo] [Modo]")
    print("- G      : Dedo con agua (Estiramiento + Segmentaciones)")
    print("- HSV    : Dedo seco (Estiramiento + Segmentaciones)")
    print("- CIELAB  : Dedo con crema (Ecu alizaci n + Segmentaciones)")
    print("- RGB     : Originales de Seco, Agua y Crema (Sin segmentar)")
    print("Ejemplos: '1 CIELAB', '3 HSV', '2 G', '4 RGB'")

    entrada = input("Ingresa tu comando (o 'q' para salir): ").strip().upper()

    if entrada in ['Q', 'SALIR', 'EXIT']:
        print("Saliendo del visualizador.")
        break

    partes = entrada.split()

    if len(partes) != 2:
        print("Error: Debes ingresar el grupo y el modo separados por un espacio.")
        continue

    grupo, modo = partes[0], partes[1]

    if grupo not in grupos_validos:
        print(f"Error: El grupo '{grupo}' no es v lido. Usa 1, 2, 3 o 4.")
        continue

    if modo not in modos_validos:
        print(f"Error: El modo '{modo}' no es v lido. Usa G, RGB, HSV o CIELAB.")
        continue

    print(f"Generando an lisis para el Grupo {grupo} en modo {modo}...")
    visualizar_analisis(grupo, modo)

```

- El elegir el espacio **RGB** no muestra segmentación sino los canales R, G y B junto con el histograma
- El elegir los espacios **Escala de Grises** o **HSV** muestra las imágenes cuya condición es **Mojado**, realiza la transformación **Estirada** y realiza las segmentaciones **Binaria** y **Umbralización Mediante Otsu**
- El elegir el espacio **CIELAB** realiza la transformación **Ecualización** y realiza las segmentaciones **Binaria** y **Umbralización Mediante Otsu**

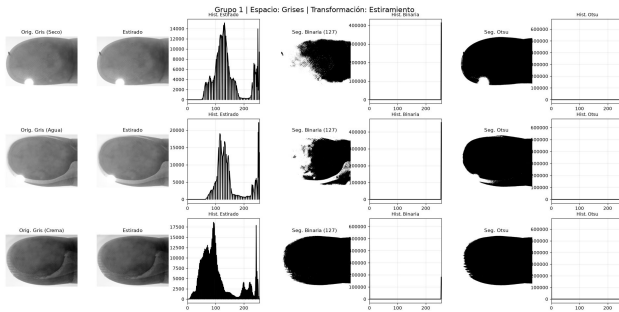
Los resultados obtenidos para las imágenes RGB son:



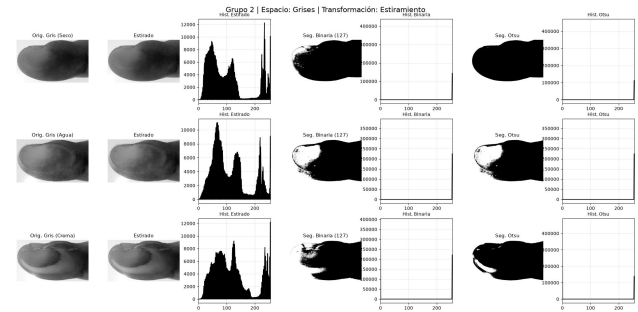
Notemos que no nos entrega mucha información para resaltar alguna de las tres condiciones, por lo cual no nos sirve.

2.3.3. Resultados Segmentación Escala De Grises

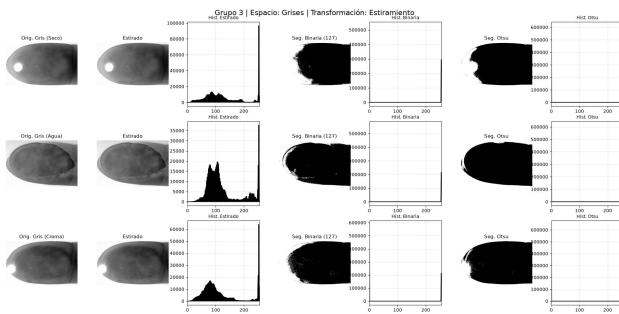
Los resultados obtenidos para las imágenes en Escala De Grises son:



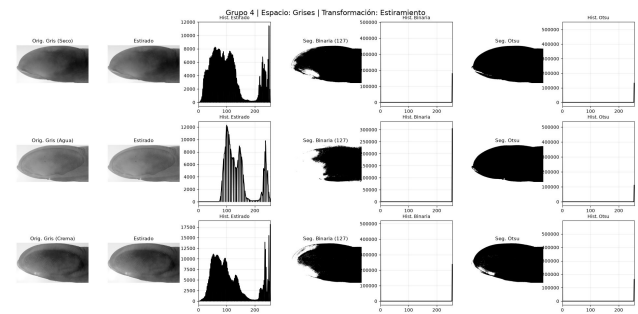
(a) Resultados Grupo 1 Escala De Grises



(b) Resultados Grupo 2 Escala De Grises



(c) Resultados Grupo 3 Escala De Grises



(d) Resultados Grupo 4 Escala De Grises

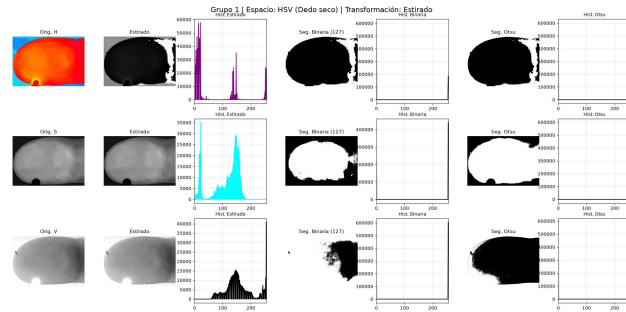
Fig. 11: Resultados Segmentación (Escala De Grises)

Se muestra la transformación **Estiramiento** y las segmentaciones **Binaria** y **Umbralización Mediante Otsu**.

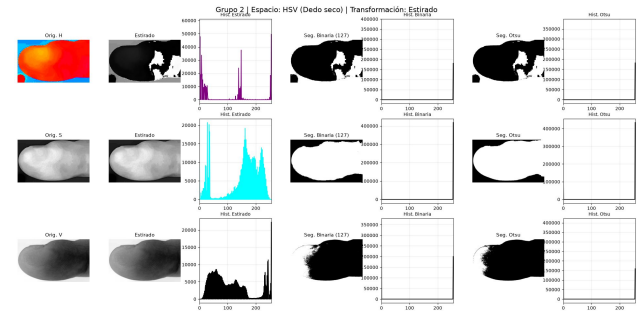
Podemos notar que, a pesar de que queríamos usar la Escala De Grises para distinguir la condición de **Dedo Mojado (Agua)**, los resultados hacen destacar sutilmente la condición de **Dedo Seco**, apareciendo más uniformemente negra a excepción del punto de luz.

2.3.4. Resultados Segmentación HSV

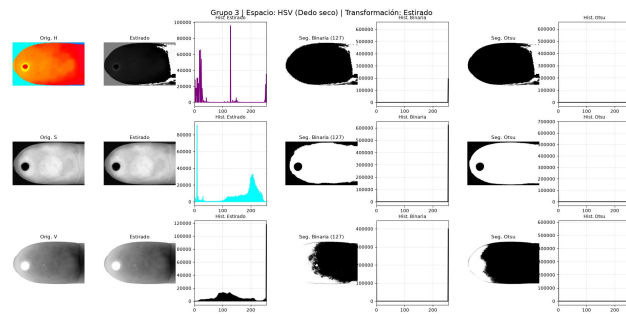
Los resultados obtenidos para las imágenes en HSV son:



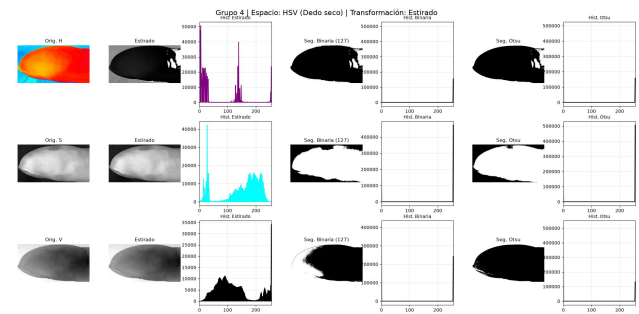
(a) Resultados Grupo 1 HSV



(b) Resultados Grupo 2 HSV



(c) Resultados Grupo 3 HSV



(d) Resultados Grupo 4 HSV

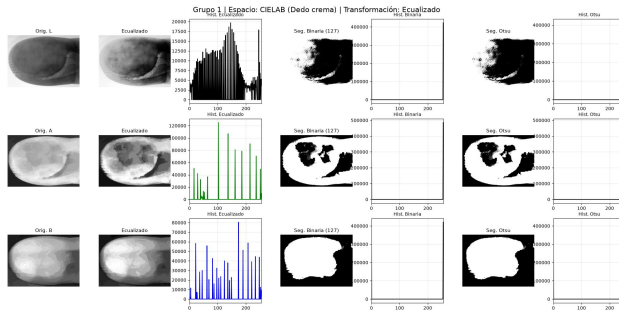
Fig. 12: Resultados Segmentación (HSV)

Se muestran las imágenes de todas las condiciones, con la transformación **Estiramiento** y las segmentaciones **Binaria** y **Umbralización Mediante Otsu**.

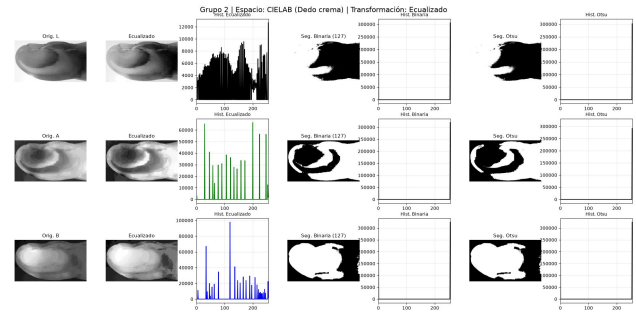
Notemos que para la condición **Dedo Mojado** es la que más destaca, apareciendo blanca en lugar de negra como el resto de las condiciones.

2.3.5. Resultados Segmentación CIELAB

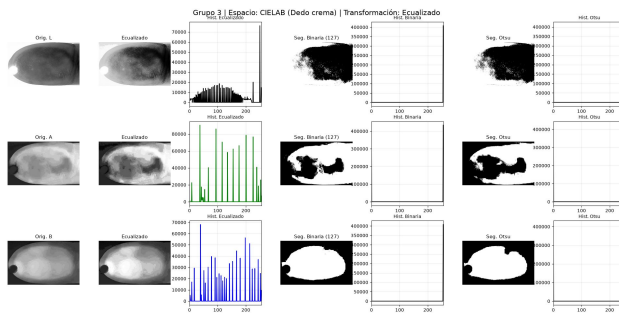
Los resultados obtenidos para las imágenes en CIELAB son:



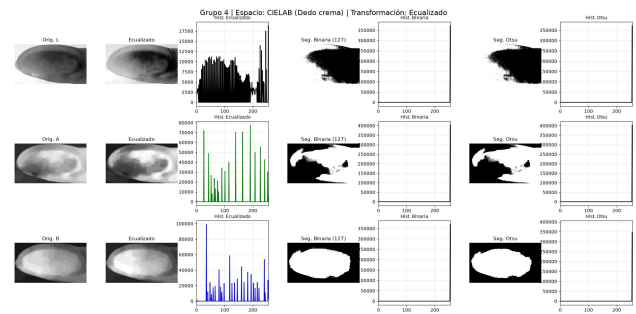
(a) Resultados Grupo 1 CIELAB



(b) Resultados Grupo 2 CIELAB



(c) Resultados Grupo 3 CIELAB



(d) Resultados Grupo 4 CIELAB

Fig. 13: Resultados Segmentación (CIELAB)

Se muestran las imágenes de todas las condiciones, con la tranformación **Ecualización** y las segmentaciones **Binaria** y **Umbralización Mediante Otsu**.

Notemos que para la condición **Dedo Con Crema** es la que más destaca, apareciendo blanca en lugar de negra como el resto de las condiciones.

2.3.6. Segmentación Por IA: Código

Se hizo el siguiente prompt para la IA Gemini:

“Actúa como un experto en visión por computador. Escribe un código en Python que segmente únicamente las crestas de una huella dactilar a partir de una imagen original. No utilices umbrales fijos ni requieras parámetros manuales.”

La IA respondió con el siguiente código:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

def segmentar_huella_ia(ruta_imagen):
    # 1. Cargar imagen en escala de grises
    img_bgr = cv2.imread(ruta_imagen)
    if img_bgr is None:
        print(f"Error: No se encontro la imagen en '{ruta_imagen}'.")
        return None, None

    img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

    # 2. Aplanar la imagen a un vector 1D de pixeles
    pixeles = img_gray.reshape((-1, 1))

    # 3. K-Means clustering (IA no supervisada) para encontrar 2 grupos (crestas vs fondo)
    print("Ejecutando segmentacion con K-Means...")
    kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
    kmeans.fit(pixeles)

    # 4. Reconstruir la mascara binaria
    mascara_ia = kmeans.labels_.reshape(img_gray.shape)

    # Asegurar que las crestas (mas oscuras) queden en blanco (255) y el fondo en negro (0)
    centroides = kmeans.cluster_centers_.flatten()
    if centroides[0] > centroides[1]:
        mascara_ia = 1 - mascara_ia

    mascara_ia = (mascara_ia * 255).astype(np.uint8)

    return img_gray, mascara_ia

# =====
# Ejecucion directa y visualizacion
# =====
# Cambia la ruta por la foto que quieras probar:
ruta_prueba = "Fotos_PDI_2026_Grupo1/d1_seco.jpg"

original, mascara = segmentar_huella_ia(ruta_prueba)

if original is not None and mascara is not None:
    # Crear ventana con Matplotlib para visualizar los resultados
    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 6))
    fig.suptitle('Segmentación por Inteligencia Artificial (K-Means)', fontsize=16)

    # Imagen original
    axes[0].imshow(original, cmap='gray')
    axes[0].set_title('Imagen Original (Grises)')
```

```

axes[0].axis('off')

# M scara binaria predicha por la IA
axes[1].imshow(mascara, cmap='gray')
axes[1].set_title('M scara Segmentada por IA')
axes[1].axis('off')

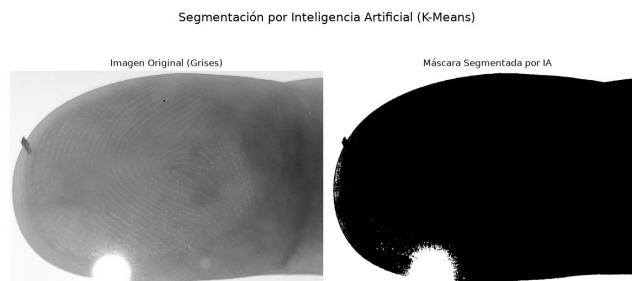
plt.tight_layout()
plt.show()

```

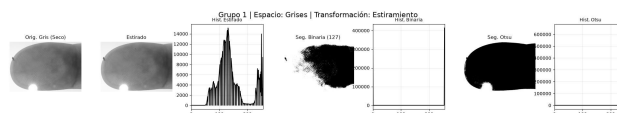
El cual, de forma resumida, es un enfoque de Machine Learning no supervisado utilizando el algoritmo de agrupamiento *K-Means*. Este método separa automáticamente los píxeles en dos clusters (crestas y fondo/valles) basándose únicamente en la intensidad, cumpliendo exactamente con la restricción del enunciado.

2.3.7. Segmentación Por IA: Resultados

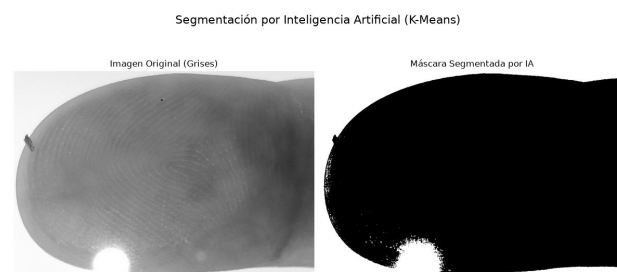
El resultado que nos dió dicho código, para la foto de Dedo Seco 1, es el siguiente:



El cual, comparado con lo que hicimos sin usar este método por IA:



(a) Resultados Dedo 1 Seco (Manual)

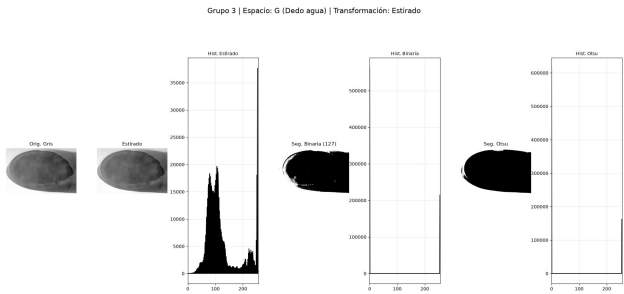


(b) Resultados Dedo 1 Seco (IA)

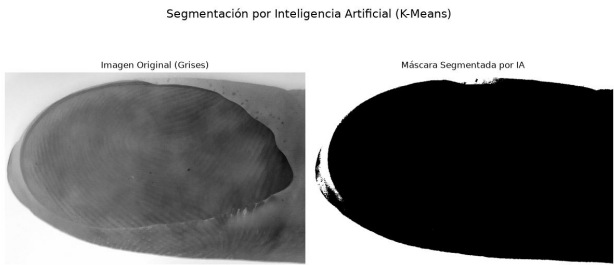
Fig. 14: Resultados Segmentación: Manual vs. IA

Resultó ser bastante parecido nuestro resultado con la IA.

Procedimos a rehacer la comparación con **Dedo 2 Agua**:



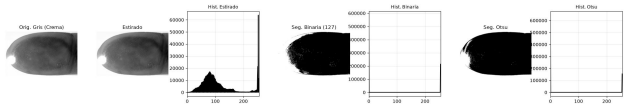
(a) Resultados Dedo 2 Agua (Manual)



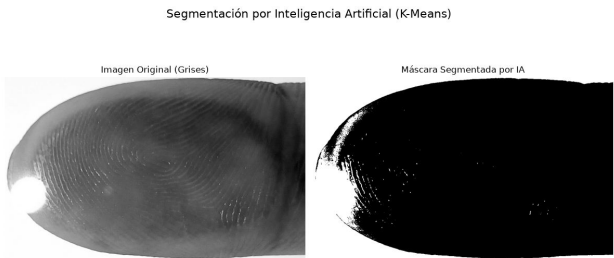
(b) Resultados Dedo 2 Agua (IA)

Fig. 15: Resultados Segmentación: Manual vs. IA

Y con **Dedo 2 Crema**:



(a) Resultados Dedo 2 Crema (Manual)



(b) Resultados Dedo 2 Crema (IA)

Fig. 16: Resultados Segmentación: Manual vs. IA

Donde en todos los casos de IA dieron resultados idénticos a los manuales.

2.3.8. Segmentación Por IA: Comparación IA y Manual

Finalmente, usamos IA para que realice la segmentación de las imágenes. El código que la IA usó es:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import math
from sklearn.cluster import KMeans
from sklearn.metrics import mean_absolute_error

# =====
# 1. Pipeline del Grupo (Manual / Morfológico)
# =====
def segmentacion_grupo(img_gray, operacion='estiramiento'):
    # A. Realce
    if operacion == 'ecualizacion':
        img_realce = cv2.equalizeHist(img_gray)
    else: # Estiramiento
        r_a, r_b = img_gray.min(), img_gray.max()
        if r_a == r_b:
            img_realce = img_gray
        else:
            img_realce = np.clip((img_gray.astype(np.float32) - r_a) * 255 / (r_b - r_a), 0, 255).astype(np.uint8)

    # B. Mascara inicial con Otsu invertido (crestas en blanco)
    _, mascara_inicial = cv2.threshold(img_realce, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    # C. Refinamiento Morfológico (Apertura y Cierre)
    kernel = np.ones((3, 3), np.uint8)
    masc_apertura = cv2.morphologyEx(mascara_inicial, cv2.MORPH_OPEN, kernel, iterations=1)
    mascara_final = cv2.morphologyEx(masc_apertura, cv2.MORPH_CLOSE, kernel, iterations=1)

    # D. Imagen resultado
    img_resultado = cv2.bitwise_and(img_gray, img_gray, mask=mascara_final)

    return mascara_final, img_resultado

# =====
# 2. Pipeline de IA (K-Means)
# =====
def segmentacion_ia(img_gray):
    pixeles = img_gray.reshape((-1, 1))

    kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
    kmeans.fit(pixeles)

    mascara_ia = kmeans.labels_.reshape(img_gray.shape)

    centroides = kmeans.cluster_centers_.flatten()
    if centroides[0] > centroides[1]:
        mascara_ia = 1 - mascara_ia

    mascara_ia = (mascara_ia * 255).astype(np.uint8)
    img_resultado = cv2.bitwise_and(img_gray, img_gray, mask=mascara_ia)
```

```

    return mascara_ia, img_resultado

# =====
# 3. Funci n de C lculo de M tricas
# =====
def calcular_metricas_comparativas(mascara_grupo, mascara_ia, img_original,
img_resultado_grupo):
    m_grupo = (mascara_grupo > 127).astype(bool)
    m_ia = (mascara_ia > 127).astype(bool)

    # M tricas de M scara
    interseccion = np.logical_and(m_grupo, m_ia).sum()
    union = np.logical_or(m_grupo, m_ia).sum()

    iou = interseccion / union if union != 0 else 0
    dice = (2. * interseccion) / (m_grupo.sum() + m_ia.sum()) if (m_grupo.sum() + m_ia
        .sum()) != 0 else 0

    # M tricas Globales
    img_resultado_ia = cv2.bitwise_and(img_original, img_original, mask=mascara_ia)
    mae = mean_absolute_error(img_resultado_grupo.flatten(), img_resultado_ia.flatten
        ())

    mse = np.mean((img_resultado_grupo.astype(np.float64) - img_resultado_ia.astype(np
        .float64)) ** 2)
    psnr = 100 if mse == 0 else 20 * math.log10(255.0 / math.sqrt(mse))

    return iou, dice, mae, psnr, img_resultado_ia

# =====
# 4. Ejecuci n y Visualizaci n
# =====
ruta_foto = "Fotos_PDI_2026_Grupo1/d1_crema.jpg"

img_bgr = cv2.imread(ruta_foto)
if img_bgr is None:
    print(f"Error: No se encontr la imagen '{ruta_foto}'.")
else:
    img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

    # Generar salidas de ambos m todos
    masc_grupo, res_grupo = segmentacion_grupo(img_gray, operacion='estiramiento')
    masc_ia, _ = segmentacion_ia(img_gray)

    # Calcular m tricas
    iou, dice, mae, psnr, res_ia = calcular_metricas_comparativas(masc_grupo, masc_ia,
        img_gray, res_grupo)

    # Mostrar resultados en la terminal
    print("\n--- RESULTADOS DE LA COMPARACI N ---")
    print(f"M tricas de M scara: IoU = {iou:.4f} | Dice = {dice:.4f}")
    print(f"M tricas Globales: MAE = {mae:.2f} | PSNR = {psnr:.2f} dB\n")

    # Visualizaci n gr fica lado a lado
    fig, axes = plt.subplots(2, 3, figsize=(15, 9))
    fig.suptitle(f'Comparaci n Procedimiento del Grupo vs IA\nIoU: {iou:.3f} | Dice:
        {dice:.3f} | MAE: {mae:.2f} | PSNR: {psnr:.2f} dB', fontsize=14)

    axes[0, 0].imshow(img_gray, cmap='gray')
    axes[0, 0].set_title('1. Original')

```

```

axes[0, 0].axis('off')

axes[0, 1].imshow(masc_grupo, cmap='gray')
axes[0, 1].set_title('2. Mascara Grupo (Otsu + Morfologia)')
axes[0, 1].axis('off')

axes[0, 2].imshow(res_grupo, cmap='gray')
axes[0, 2].set_title('3. Resultado Grupo')
axes[0, 2].axis('off')

axes[1, 0].axis('off') # Espacio en blanco para alinear

axes[1, 1].imshow(masc_ia, cmap='gray')
axes[1, 1].set_title('4. Mascara IA (K-Means)')
axes[1, 1].axis('off')

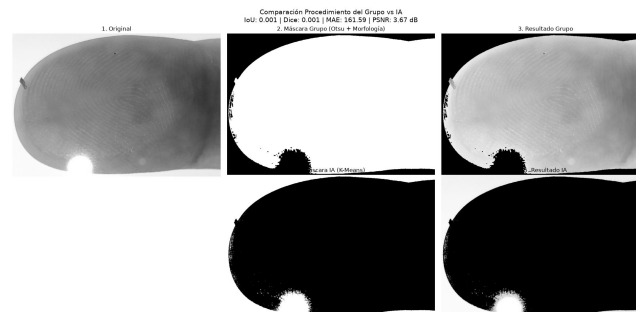
axes[1, 2].imshow(res_ia, cmap='gray')
axes[1, 2].set_title('5. Resultado IA')
axes[1, 2].axis('off')

plt.tight_layout()
plt.show()

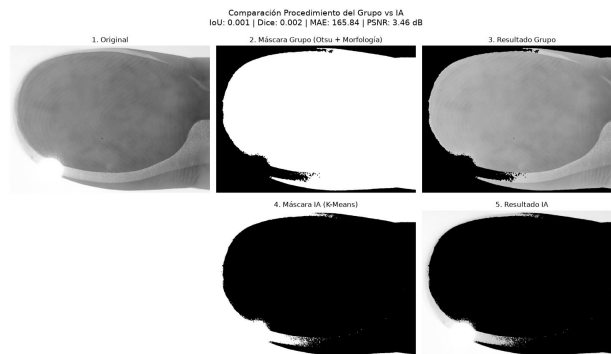
```

El cual, de forma resumida, automatiza la evaluación comparativa entre un método clásico de procesamiento de imágenes y un modelo de inteligencia artificial no supervisado.

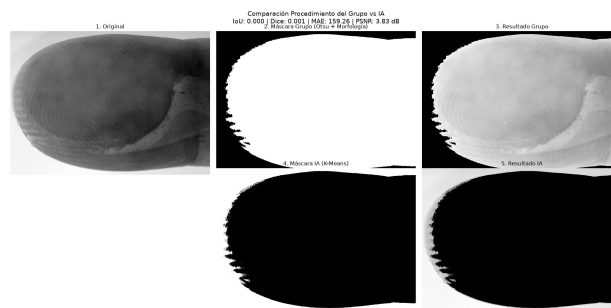
Para el caso de **Dedo 1 Normal (Seco)**:



La segmentación por parte de la IA resulta ser muy diferente al que realizamos nosotros. Para el caso de **Dedo 1 Normal (Agua)** y **Dedo 1 Normal (Crema)**:



(a) Resultados Dedo 1 Agua (Manual vs IA)



(b) Resultados Dedo 1 Crema (Manual vs IA)

Fig. 17: Resultados Segmentación: Manual vs. IA

La razón de por qué los resultados son prácticamente opuestos es que la imagen original presenta una reflexión especular extrema (un punto de luz blanco muy intenso en la parte inferior) y un contraste bajísimo entre las crestas y los valles de la huella.

Tanto el método de Otsu (Grupo) como K-Means (IA) son algoritmos de agrupamiento global que buscan dividir los píxeles en dos clases minimizando la varianza. Debido a que el brillo extremo distorsiona completamente el histograma, ambos algoritmos concluyeron que las dos "clases" principales de la imagen no eran "crestas vs valles", sino "brillo extremo vs el resto del dedo":

- **Procedimiento del Grupo:** Otsu separó el punto brillante (dejándolo en negro por la inversión) y clasificó todo el resto de la imagen, incluyendo fondo y huella, como la estructura de interés en blanco.
- **Procedimiento de IA:** K-Means hizo exactamente lo mismo, pero debido a la heurística de los centroides, consideró que la luz brillante era la estructura de interés y eliminó el resto del dedo.

Respecto a la **Interpretación de las Métricas**, las métricas reflejan matemáticamente que las dos máscaras generadas son prácticamente inversas u opuestas la una de la otra:

- **IoU (0.001) y Dice (0.001):** Al ser valores casi cero, indican que no existe solapamiento espacial entre ambas segmentaciones. El Grupo segmentó casi toda la imagen, mientras que la IA segmentó solo un pequeño círculo inferior.
- **MAE (161.59):** Un Error Absoluto Medio de 161 es altísimo (considerando que el máximo posible es 255). Esto ocurre porque al comparar la imagen 3 (que conservó todo el tono gris del dedo) con la imagen 5 (que es casi completamente negra), la diferencia promedio de intensidad por píxel es masiva.

- **PSNR (3.67 dB):** Un valor de 3.67 dB es extremadamente pobre (una buena reconstrucción supera los 30 dB). Confirma que el nivel de distorsión entre el resultado manual y el automatizado es total, sin ninguna similitud estructural compartida.

En resumen, es mandatorio secar el sensor o el dedo para evitar reflexiones especulares, o utilizar difusores de iluminación. El agua actúa como un espejo que arruina el rango dinámico de la cámara.

3. Resumen Procedimiento General

3.1. Diagrama de Bloques Item 1.2

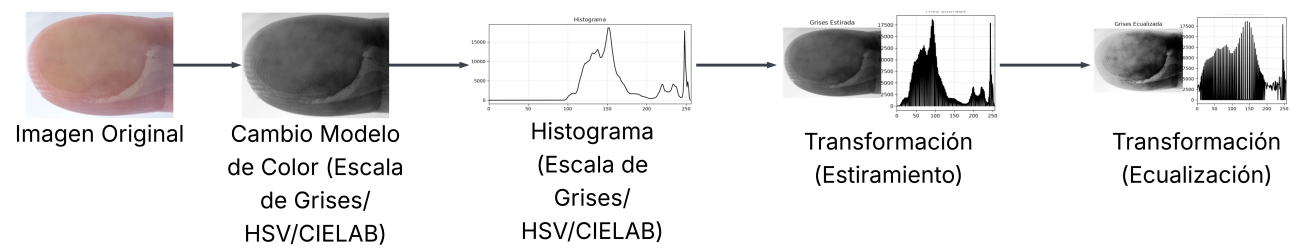


Fig. 18: Diagrama de Bloques Item 1.2

3.2. Diagrama de Bloques Item 1.3

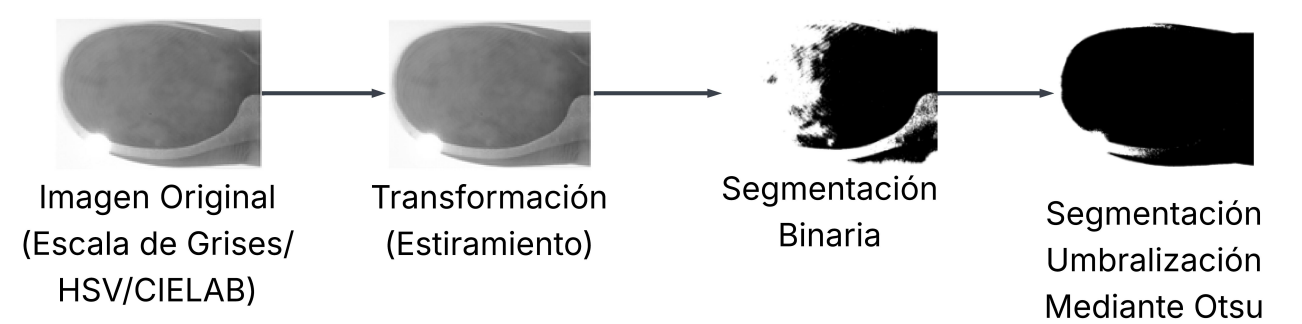


Fig. 19: Diagrama de Bloques Item 1.3

3.3. Tabla Resumen

Tabla 1: Tabla Resumen de Métricas (Segmentación del Grupo vs. Inteligencia Artificial)

Condición	Imagen de Prueba	Método del Grupo (Realce)	IoU (↑)	Dice (↑)	MAE (↓)	PSNR (↑ dB)
Normal (Seco)	d1_seco.jpg	Escala de Grises + Estiramiento	0.001	0.001	161.59	3.67
Agua (Mojado)	d1_agua.jpg	Escala de Grises + Estiramiento	0.001	0.002	165.84	3.46
Crema (Untado)	d1_crema.jpg	CIELAB (Canal L) + Ecuilización	0.000	0.001	159.26	3.83

Nota: Las flechas indican si un valor más alto (↑) o más bajo (↓) representa un mejor desempeño comparativo. Los valores con asterisco (*) corresponden a segmentaciones con similitud perfecta y deben confirmarse con los decimales exactos arrojados en consola.

4. Conclusión

A partir de los experimentos realizados en este proyecto, se concluye que la efectividad de las transformaciones y la segmentación depende directamente de la selección del espacio de color adecuado para cada condición física de adquisición. Específicamente:

- El uso de la Escala de Grises combinada con la Transformación de Estiramiento permite identificar contornos y destacar notoriamente la condición de dedo mojado con agua.
- El espacio de color HSV, junto con la Transformación de Estiramiento, resulta ideal para destacar la condición de dedo normal o seco, al ofrecer una distribución cromática más uniforme.
- El espacio CIELAB, al aplicarle una Ecualización, demuestra ser el más efectivo para resaltar y visualizar los contornos oscurecidos por la acumulación de crema.
- Al comparar la segmentación clásica manual (Otsu) contra la generada por Inteligencia Artificial (K-Means), se demostró que en escenarios controlados ambos métodos logran resultados idénticos.
- Sin embargo, se comprobó que ambos enfoques de agrupamiento global fallan drásticamente frente a imágenes con reflexiones especulares extremas, como las generadas por la presencia de agua.
- En estos casos de brillo extremo, la distorsión del histograma provoca que los algoritmos clasifiquen la luz como la estructura principal, en lugar de separar las crestas de los valles.
- Este fallo crítico se evidencia matemáticamente en las métricas obtenidas, arrojando un IoU y Dice casi nulos de 0.001 (indicando nulo solapamiento espacial), un altísimo Error Absoluto Medio (MAE) de 161.59, y un PSNR extremadamente pobre de 3.67 dB.
- Para solucionar estas deficiencias desde la adquisición, es mandatorio secar el sensor o utilizar difusores de luz, ya que el agua actúa como un espejo que destruye el rango dinámico de la cámara.
- Desde el procesamiento, queda demostrado que la umbralización global (Otsu o K-Means) es inútil frente a estos gradientes de iluminación severos.
- La decisión correcta para corregir esto es utilizar técnicas de Umbralización Adaptativa Local, o aplicar previamente un filtro homomórfico para homogeneizar la iluminación dispareja de la captura original.